Simulation Report

Topic:

Arithmetic Logic Unit (ALU) Design and Simulation

Internship Task 1 – VLSI Design

Name: Likhith Gowda H R

Tool Used: Cadence Genus, SimVision

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	**INTRODUCTION**	**1–3**
1.1	Introduction	1
1.2	Theory of Arithmetic Logic Unit	2
1.3	Objective	2
1.4	Operation Table	2
1.5	Design Methodology	3
2	**DESIGN METHODOLOGY**	**4-6**
2.1	ALU Design Implementation	4
2.2	Testbench Implementation	5
2.3	RTL / Schematic View	6
3	**SIMULATION**	**7–8**
3.1	Simulation Methodology	**7**
3.2	Simulation Results and Analysis	**7**
4	**SYNTHESIS AND PERFORMANCE ANALYSIS**	**9-12**
4.1	Power Analysis	**9**
4.2	Area Analysis	**10**
4.3	Timing Analysis	**11**
4.4	Gate-Level Report	**12**
5	**RESULTS**	**13**
6	**DISCUSSION**	**14**
7	**CONCLUSION**	**15**

LIST OF TABLES

Table No.	Title	Page No.
1.1	ALU Operation Selection Table	2

LIST OF FIGURES

Figure No.	Title	Page No.
2.1.1	Verilog Implementation of ALU	4
2.2.1	ALU Testbench Implementation	5
2.3.1	Synthesized RTL Schematic of ALU	6
3.2.1	Simulation Waveform of ALU	7
4.1.1	Power Analysis Report	8
4.2.1	Area Utilization Report	9
4.3.1	Timing Analysis Report	10
4.4.1	Gate-Level Synthesis Report	11

Chapter 1
Introduction

1.1 Introduction

The Arithmetic Logic Unit (ALU) is a fundamental component of digital systems and microprocessors responsible for performing arithmetic and logical operations. It executes operations such as addition, subtraction, AND, OR, and NOT, which are essential for computational processes in digital circuits.

In this project, a basic ALU is designed using Verilog Hardware Description Language (HDL) and verified using Cadence simulation tools. The design is tested using a testbench, and the functional correctness is verified through simulation waveforms. The synthesized design is further analyzed for performance parameters such as area utilization, power consumption, and timing characteristics.

1.2 Theory of Arithmetic Logic Unit

An Arithmetic Logic Unit is a combinational circuit that performs arithmetic and logical operations on binary inputs. It accepts input operands and a control signal that determines the operation to be performed.

Arithmetic operations include addition and subtraction, while logical operations include AND, OR, and NOT. These operations are implemented using basic logic gates and arithmetic circuits. The ALU processes the input data according to the control signal and generates the corresponding output.

In this project, a 3-bit select signal is used to choose different operations performed by the ALU.

1.3 Objective

The primary objective of this project is to design and simulate a basic Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language. The ALU is developed to perform essential arithmetic and logical operations such as addition, subtraction, AND, OR, and NOT based on a 3-bit select signal.

The project aims to understand the working of combinational logic circuits and their implementation using hardware description languages. Functional verification of the ALU is carried out using a testbench and simulation waveform analysis to ensure correct operation.

Another objective is to perform synthesis of the design and analyze performance parameters such as area utilization, power consumption, and timing characteristics. The project provides practical exposure to digital design concepts and the VLSI design flow using Cadence simulation and synthesis tools.

1.4 Operation Table

Table 1.4: ALU Operation Selection Table

Sel	Operation
000	ADDITION
001	SUBTRACTION
010	AND
011	OR
100	NOT

The ALU operation table 1 defines the functionality of the circuit based on the 3-bit select signal (sel). The select signal determines which arithmetic or logical operation will be executed by the ALU. Each binary value of the select signal corresponds to a specific operation.

When sel = 000, the ALU performs addition of inputs A and B. When sel = 001, subtraction is performed. For logical operations, sel = 010 performs bitwise AND, sel = 011 performs bitwise OR, and sel = 100 performs bitwise NOT operation on input A.

This selection mechanism allows the ALU to perform multiple operations using a single hardware module, making it efficient and flexible for digital system implementation.

1.5 Design Methodology

The ALU is implemented using Verilog HDL. The design accepts two 4-bit input operands A and B and a 3-bit select signal that determines the operation to be performed. The output result represents the outcome of the selected operation.

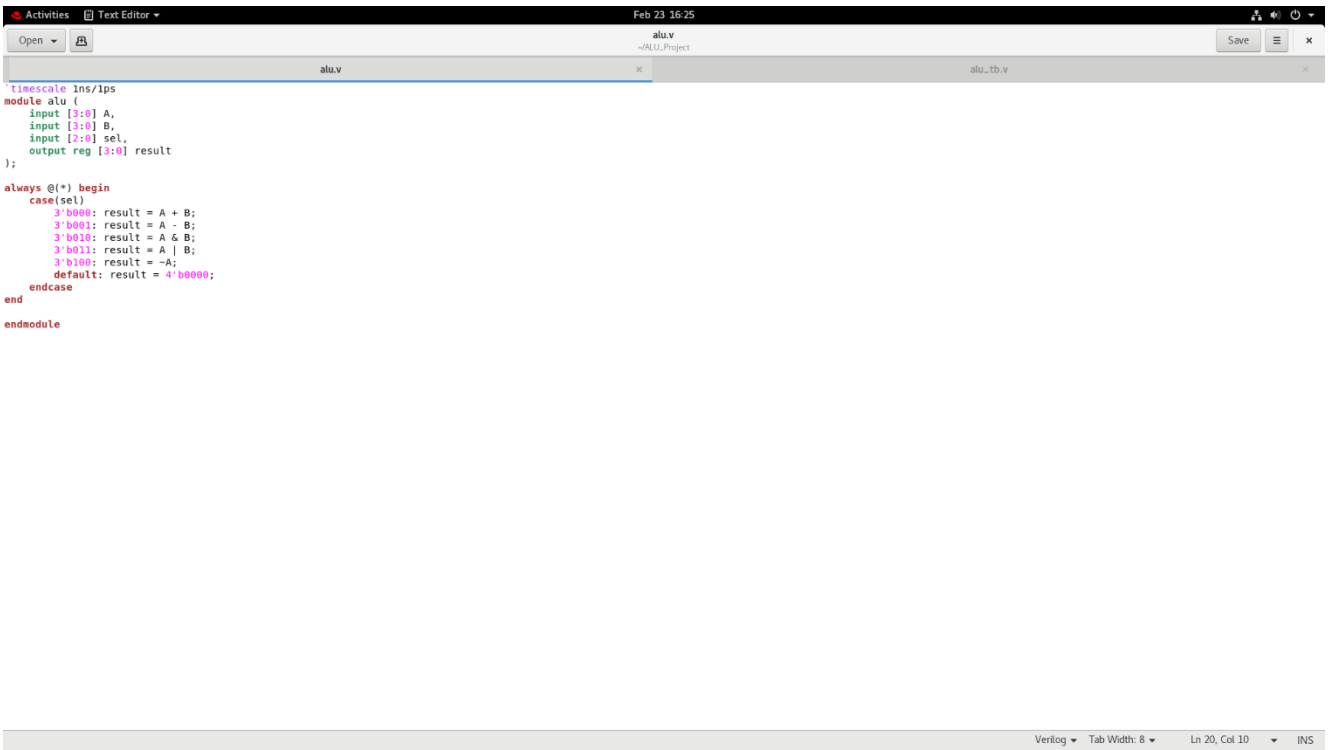
The design uses combinational logic implemented through an always block and case statement. Based on the select signal, the corresponding arithmetic or logical operation is executed. A default condition is included to ensure stable output behavior.

Chapter 2

DESIGN METHODOLOGY

2.1 ALU Design Implementation:

The ALU module was coded in Verilog using behavioral modeling. The design performs arithmetic and logical operations depending on the select signal.



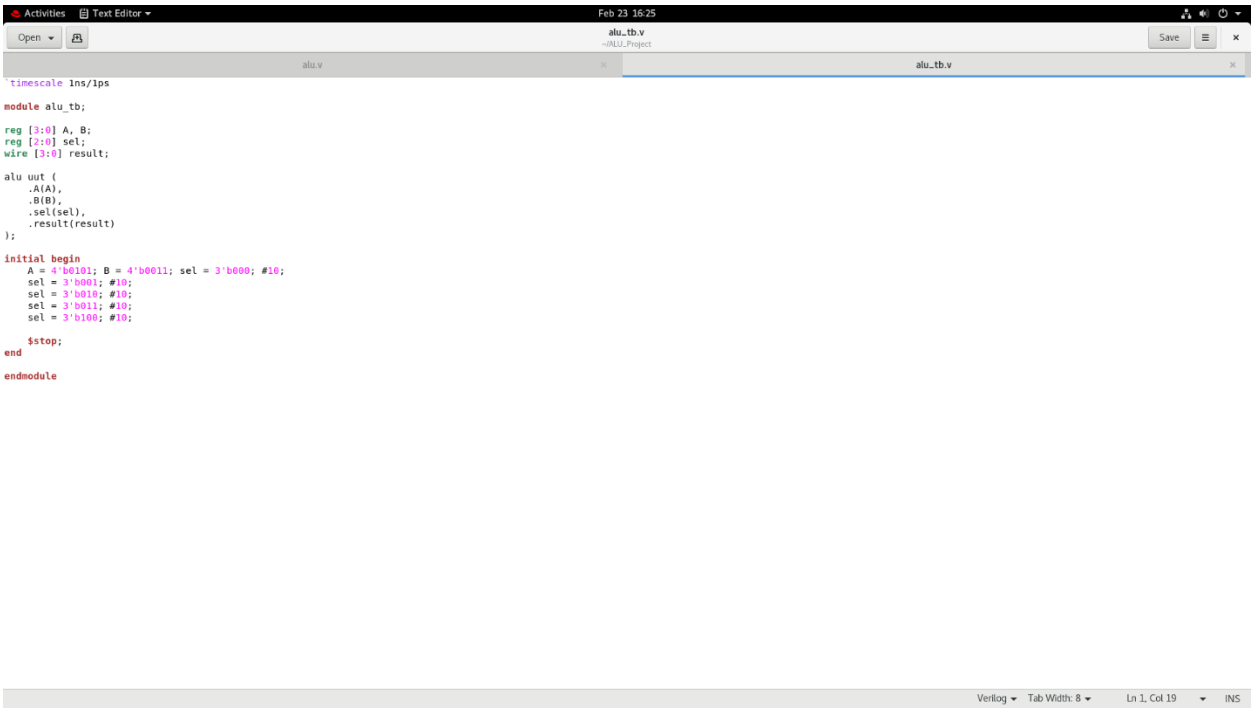
```
`timescale 1ns/1ps
module alu (
    input [3:0] A,
    input [3:0] B,
    input [2:0] sel,
    output reg [3:0] result
);

always @(*) begin
    case(sel)
        3'b000: result = A + B;
        3'b001: result = A - B;
        3'b010: result = A & B;
        3'b011: result = A | B;
        3'b100: result = ~A;
        default: result = 4'b0000;
    endcase
end

endmodule
```

Figure 2.1.1: Verilog Implementation of ALU

Figure 2.1.1 shows the Verilog implementation of the Arithmetic Logic Unit (ALU). The module takes two 4-bit inputs, A and B, and a 3-bit select signal that determines the operation to be performed. Based on the select value, the ALU executes arithmetic operations such as addition and subtraction, and logical operations such as AND, OR, and NOT. The result is generated as a 4-bit output. The design is implemented using a case statement to ensure proper selection of operations.

2.2 Testbench Implementation:

A testbench was developed to verify the functionality of the ALU. It applies different input combinations to test arithmetic and logical operations and observes the output response.



```
`timescale 1ns/1ps

module alu_tb;

reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] result;

alu uut (
    .A(A),
    .B(B),
    .sel(sel),
    .result(result)
);

initial begin
    A = 4'b0101; B = 4'b0011; sel = 3'b000; #10;
    sel = 3'b001; #10;
    sel = 3'b010; #10;
    sel = 3'b011; #10;
    sel = 3'b100; #10;

    $stop;
end

endmodule
```

Figure 2.2.1: ALU Testbench Implementation

Figure 2.2.1 shows the Verilog testbench developed to verify the functionality of the ALU design. The testbench provides different input values for operands A and B along with various select signals to test arithmetic and logical operations. It applies input stimuli and observes the output results during simulation to ensure that the ALU performs the required operations correctly.

2.3 RTL / Schematic View:

After synthesis, the RTL schematic was generated to visualize the hardware implementation of the ALU design. The schematic represents internal logic gates and arithmetic units.

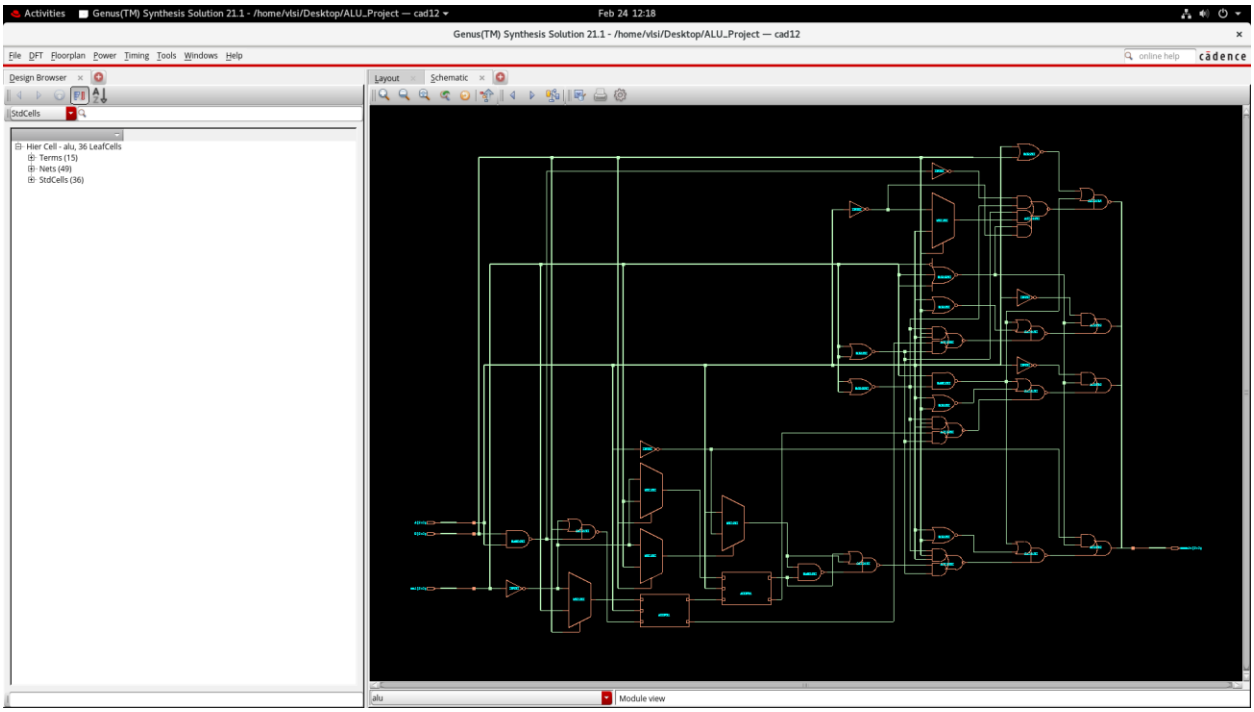


Figure 2.3.1: Synthesized RTL Schematic of ALU

Figure 2.3.1 shows the synthesized RTL schematic of the ALU generated after synthesis using Cadence tools. The schematic represents the internal hardware structure of the design, including logic gates and arithmetic components used to perform different operations. It verifies that the behavioral Verilog code has been successfully converted into hardware-level implementation.

Chapter 3

Simulation

3.1 Simulation Methodology

The ALU design and testbench were simulated using Cadence simulation tools. The design was compiled and executed to verify functionality. Simulation waveforms were generated using SimVision to observe input and output signals.

Different input combinations were applied, and the results were compared with expected theoretical values.

3.2 Simulation Results and Analysis:

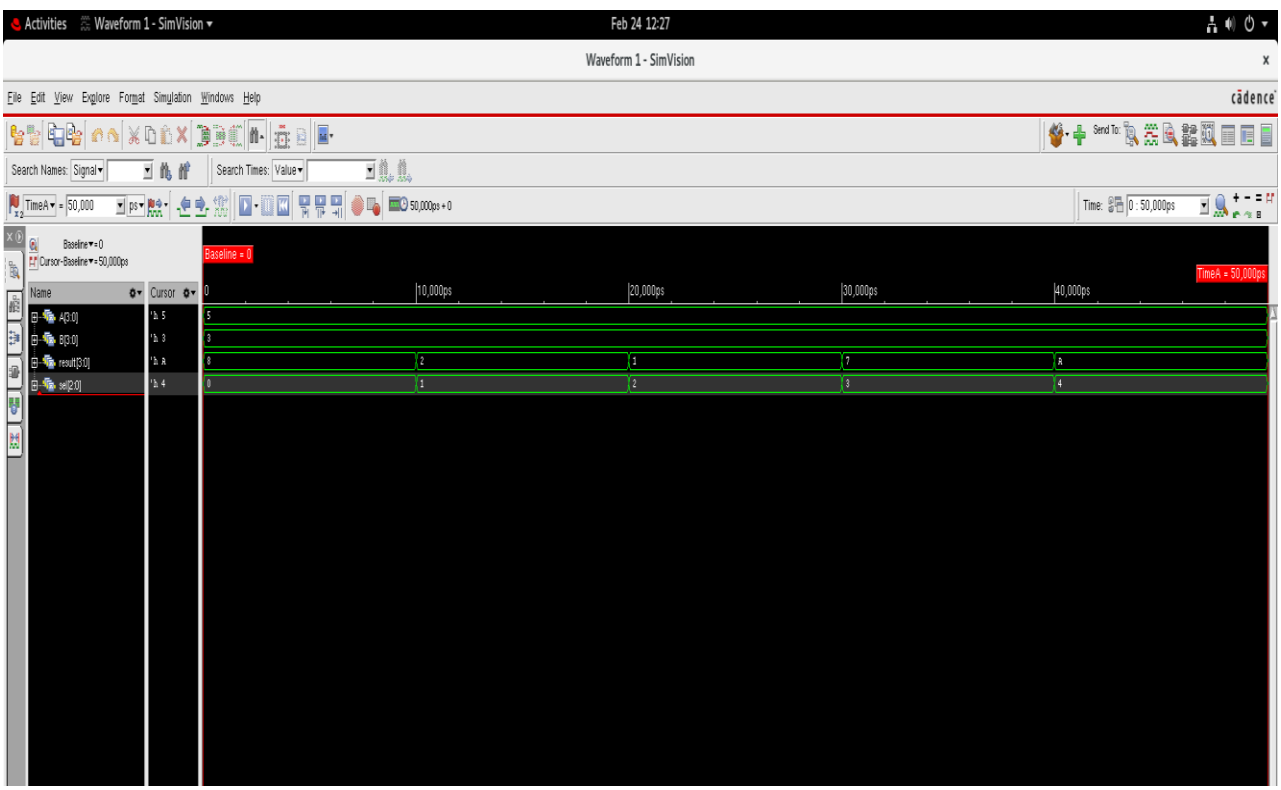


Figure 3.2.1: Simulation Waveform of ALU

The simulation results of the ALU were verified using the SimVision waveform viewer, as shown in Figure 3.2.1. The waveform displays the input signals A [3:0], B [3:0], select signal sel [2:0], and the output result [3:0] over time. For the applied inputs A = 5 and B = 3, the output changes according to the select signal, performing addition when sel = 000, subtraction when sel = 001, bitwise AND when sel = 010, bitwise OR when sel = 011, and NOT operation when sel = 100. The observed outputs match the expected theoretical results, confirming the correct functionality and operational accuracy of the ALU design.

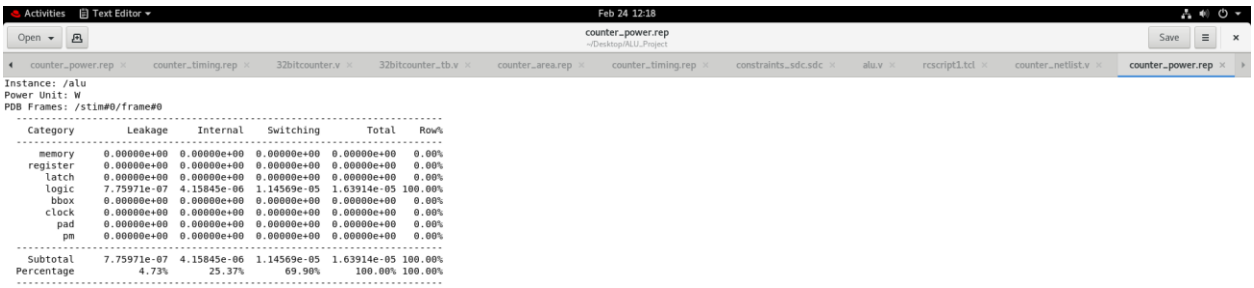
Chapter 4

Synthesis And Performance Analysis

After simulation, the design was synthesized using Cadence Genus to evaluate hardware performance.

4.1 Power Analysis

The power report provides information about dynamic and leakage power consumption of the design.



```
Instance: /alu
Power Unit: W
PDB Frames: /stim#0/frame#0
  ----------------------------------------------------------------------------
    Category          Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic     7.75971e-07  4.15845e-06  1.14569e-05  1.63914e-05 100.00%
         bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ----------------------------------------------------------------------------
    Subtotal     7.75971e-07  4.15845e-06  1.14569e-05  1.63914e-05 100.00%
  Percentage           4.73%       25.37%       69.90%     100.00% 100.00%
  ----------------------------------------------------------------------------
```

Figure 4.1.1: Power Analysis Report

Figure 4.1.1 shows the power analysis report of the synthesized ALU design generated using Cadence Genus. The report provides information about the power consumption of the circuit, including dynamic power due to switching activity and leakage power caused by static current. Power analysis is an important parameter in VLSI design as it determines the energy efficiency of the system. The results indicate that the ALU design operates with acceptable power consumption, demonstrating efficient implementation of arithmetic and logical operations.

4.2 Area Analysis

The area report shows the number of logic cells and hardware resources used.

```
Activities   Text Editor ▾                    Feb 24 12:18
Open ▾                    counter_area.rep                    Save
                          ~/Desktop/ALU_Project
32bitcounter.v   32bitcounter_tb.v   counter_area.rep   counter_timing.rep   constraints_sdc.sdc   alu.v   rcscript1.tcl   counter_netlist.v   counter_power.rep   counter_timing.rep   counter_area.rep

============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Feb 24 2026  12:17:59 pm
  Module:                 alu
  Technology library:     slow
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------
alu                     36    206.634     0.000      206.634 <none> (D)
  (D) = wireload is default in technology library
```

Figure 4.2.1: Area Utilization Report

Figure 4.2.1 presents the area utilization report of the synthesized ALU design. The report shows the total number of standard cells and logic elements used along with the overall area occupied by the circuit. Area analysis is a crucial factor in VLSI design because it directly affects chip size, fabrication cost, and hardware complexity. The synthesis results indicate that the ALU is implemented using an optimized number of logic cells, ensuring efficient utilization of hardware resources while maintaining correct functionality.

4.3 Timing Analysis

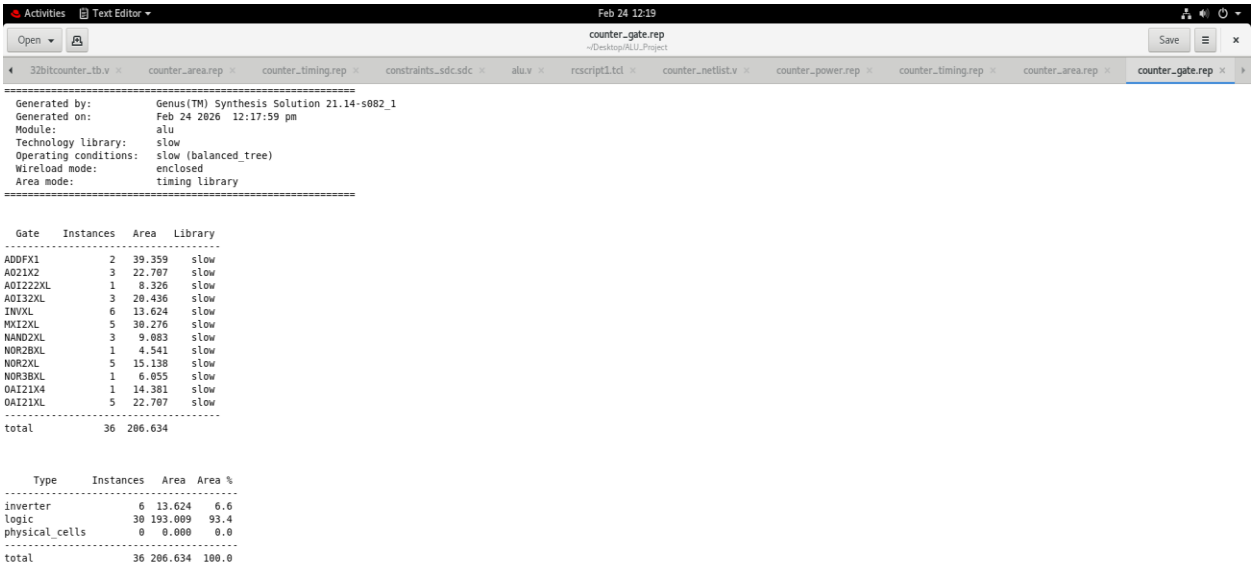
Timing analysis evaluates propagation delay and performance of the circuit.

Figure 4.3.1: Timing Analysis Report

Figure 4.3.1 shows the timing analysis report of the synthesized ALU design. The timing report provides information about propagation delay, critical path delay, and overall performance of the circuit. Timing analysis is essential in digital design to ensure that the circuit operates correctly within required speed constraints. The results indicate that the ALU meets timing requirements, confirming reliable and stable operation of the design under the specified conditions.

4.4 Gate-Level Report

The gate-level report shows the number of logic gates used in implementation.



```
Activities   Text Editor                                   Feb 24 12:19
Open                          counter_gate.rep                                   Save
                              ~/Desktop/ALU_Project
32bitcounter_tb.v | counter_area.rep | counter_timing.rep | constraints_sdc.sdc | alu.v | rcscript1.tcl | counter_netlist.v | counter_power.rep | counter_timing.rep | counter_area.rep | counter_gate.rep
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Feb 24 2026  12:17:59 pm
  Module:                 alu
  Technology library:     slow
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================


  Gate    Instances   Area   Library
-------------------------------------
ADDFX1            2   39.359    slow
AO21X2            3   22.707    slow
AOI222XL          1    8.326    slow
AOI32XL           3   20.436    slow
INVXL             6   13.624    slow
MXI2XL            5   30.276    slow
NAND2XL           3    9.083    slow
NOR2BXL           1    4.541    slow
NOR2XL            5   15.138    slow
NOR3BXL           1    6.055    slow
OAI21X4           1   14.381    slow
OAI21XL           5   22.707    slow
-------------------------------------
total            36  206.634


     Type      Instances   Area  Area %
-----------------------------------------
inverter               6  13.624    6.6
logic                 30 193.009   93.4
physical_cells         0   0.000    0.0
-----------------------------------------
total                 36 206.634  100.0
```

Figure 4.4.1: Gate-Level Synthesis Report

Figure 11.4.1shows the gate-level synthesis report of the ALU design generated after synthesis. The report provides details about the number and types of logic gates used in the hardware implementation of the circuit. Gate-level analysis helps evaluate the hardware complexity and structural efficiency of the design. The results indicate that the ALU is implemented using an optimized number of logic gates, ensuring efficient hardware realization and reliable circuit performance.

Chapter 5

Results

The simulation results clearly confirm the correct functional behavior of the Arithmetic Logic Unit for all defined operations. For different values of the select signal, the ALU successfully performed addition, subtraction, bitwise AND, bitwise OR, and NOT operations as expected. The waveform analysis verified that the output results matched the theoretical calculations for the given input combinations.

Furthermore, synthesis analysis provided insight into the hardware performance of the design. The area report indicated efficient utilization of logic cells and standard cells, demonstrating optimized hardware implementation. The power analysis showed acceptable dynamic and leakage power consumption, confirming energy-efficient design behavior. Timing analysis verified that the circuit met required timing constraints without violations, ensuring reliable operation. Overall, the results validate both the functional correctness and structural efficiency of the ALU design.

Chapter 6

Discussion

This project demonstrates the practical implementation of arithmetic and logical operations using combinational digital logic. By describing the ALU in Verilog HDL, the behavioral functionality was successfully translated into a hardware-level implementation through synthesis. The use of a case statement for operation selection allowed flexible control of multiple functions within a single module.

The verification process through simulation ensured that all operations produced correct outputs before synthesis, reducing potential design errors. The synthesis results further confirmed that the design is hardware-efficient and meets performance requirements in terms of area, power, and timing. This approach reflects the standard VLSI design flow, where functional verification is followed by performance evaluation.

The project strengthens understanding of hardware description languages, combinational logic design, simulation techniques, and synthesis optimization. It also demonstrates how digital circuits are transformed from high-level code to physical hardware representation.

Chapter 7

Conclusion

The Arithmetic Logic Unit (ALU) was successfully designed, implemented, simulated, and synthesized using Verilog HDL and Cadence tools. The design correctly performs arithmetic operations such as addition and subtraction, as well as logical operations including AND, OR, and NOT, based on a 3-bit select signal.

Functional verification through simulation waveforms confirmed accurate operation for all test cases. The synthesis analysis demonstrated efficient utilization of hardware resources, acceptable power consumption, and satisfactory timing performance. No functional or timing violations were observed, indicating stable and reliable circuit behavior.

This project provided valuable hands-on experience in digital design and the VLSI development process, including coding, simulation, verification, synthesis, and performance analysis. The implemented ALU design can be further extended to support additional operations or higher bit-width architectures, making it suitable for integration into larger digital systems such as processors and embedded systems.